

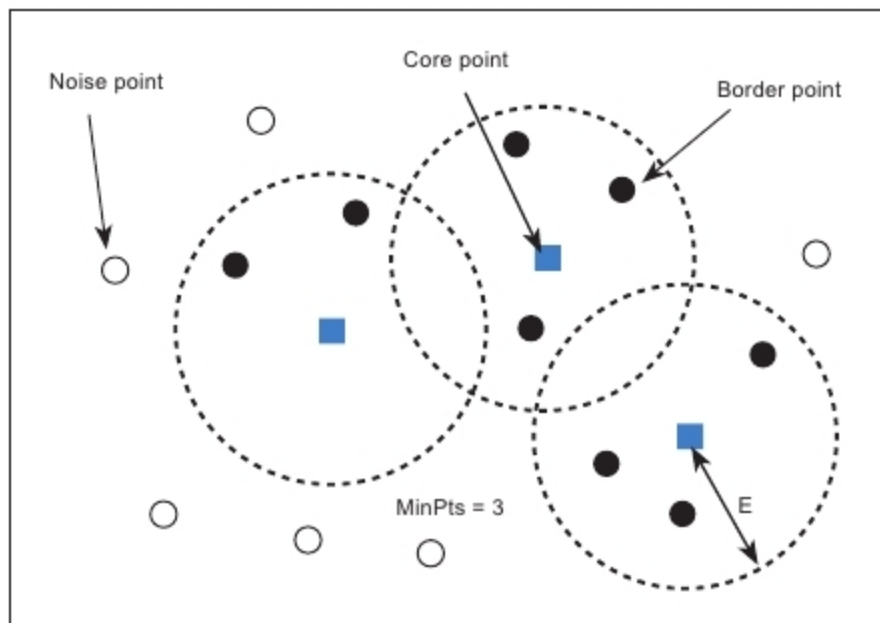


Figure 6: Data points

DBSCAN (Density Based Spatial Clustering of Applications with Noise) is a specific and very popular type of density based clustering, which is particularly effective for tasks like class identification in a spatial context, and it can find out any arbitrary shaped cluster without getting affected by noise. Also, it is efficient for large databases.

How DBSCAN works

DBSCAN works on the idea that if a particular point belongs to a cluster, it should be near lots of other points in that cluster. It works based on two parameters — radius and minimum points. R or 'eps' determines a specified radius which, if it includes enough points within it, is called a dense area. M determines the minimum number of data points we want in a neighbourhood to define a cluster. As a general rule, the minimum M can be derived from the number of dimensions D in the data set as, $M \geq D+1$. The minimum value of M that can be chosen is 3.

In this algorithm, we have the following three types of data points.

Core point: A point is a core point if it has more than one minimum point, M , within ϵ s.

Border point: This is a point which has fewer than the minimum points within the radius (ϵ s) but is in the neighbourhood of a core point.

Noise or outlier: A point that is not a core or border point. Epsilon ϵ or, in short, ϵ s, indicates the radius.

The DBSCAN algorithm can be abstracted in the following steps:

1. The algorithm starts with an arbitrary point which has not been visited. And if this point contains MinPts within the neighbourhood, cluster formation starts. Otherwise the point is labelled as noise.
2. For each core point, if it is not already assigned to a cluster, create a new cluster.
3. Identify and assign border points to their respective core points.

4. Iterate through the remaining unvisited points in the data set. Points that do not belong to any cluster are noise.

The main drawback of DBSCAN is that it doesn't perform as well as others when the clusters are of varying density. This is because the setting of the distance threshold ϵ and MinPts for identifying the neighbourhood points varies from cluster to cluster, when the density varies. If the data and features are not so well understood by a domain expert, then setting up and MinPts could be tricky. Meanwhile, the DBSCAN algorithm overcomes the drawbacks of k-means, such as k-means forms spherical clusters only, is sensitive towards outliers and requires one to specify the number of clusters.

The DBSCAN clustering algorithm is demonstrated below:

```
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn import metrics
from sklearn.datasets.samples_generator import make_blobs
from sklearn.preprocessing import StandardScaler

#####
### #Generate sample data
centers = [[1, 1], [-1, -1], [1, -1]]
X, labels_true = make_blobs(n_samples=750, centers=centers,
                           cluster_std=0.4,
                           random_state=0)

X = StandardScaler().fit_transform(X)

#####
# Compute DBSCAN
db = DBSCAN(eps=0.3, min_samples=10).fit(X)
core_samples_mask = np.zeros_like(db.labels_, dtype=bool)
core_samples_mask[db.core_sample_indices_] = True
labels = db.labels_

#####
# Number of clusters in labels, ignoring noise if present.
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = list(labels).count(-1)

print('Estimated number of clusters: ', n_clusters)
print('Estimated number of noise points: ', n_noise)
print("Homogeneity: %0.3f" % metrics.homogeneity_score(labels_true, labels))
print("Completeness: %0.3f" % metrics.completeness_score(labels_true, labels))
print("V-measure: %0.3f" % metrics.v_measure_score(labels_true, labels))
print("Adjusted Rand Index: %0.3f" % metrics.adjusted_rand_score(labels_true, labels))
print("Adjusted Mutual Information : %0.3f " % metrics.adjusted_mutual_info_score(labels_true, labels))
```